LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Rafif Fathi Mohammad 2609106041

Kelas B1 ‘26

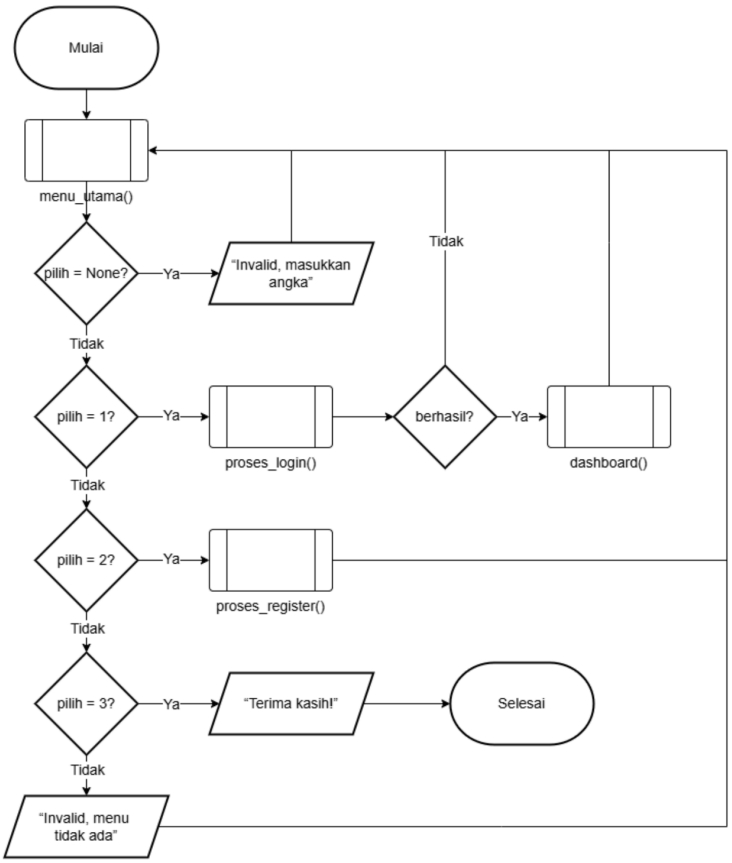
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

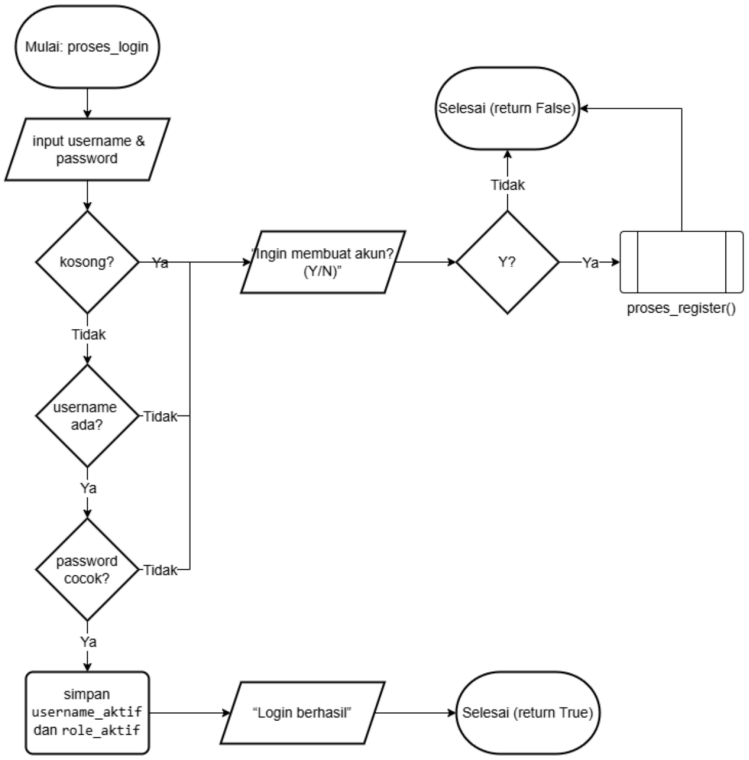
2026

1. Flowchart



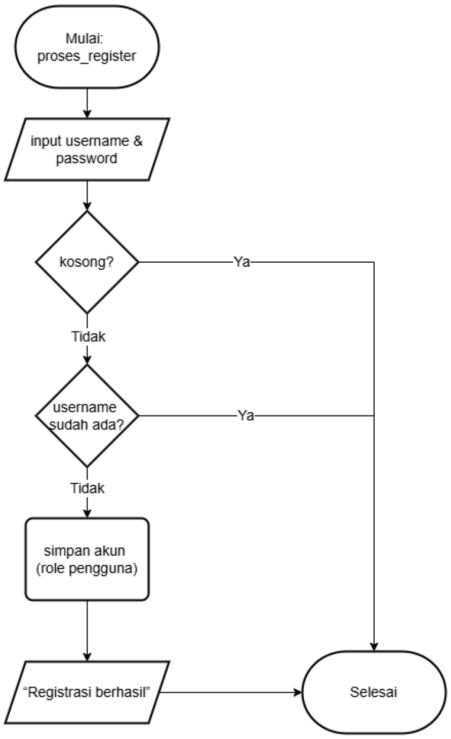
Gambar 1.1 Flowchart Utama (main dan menu_utama)

Alur program dimulai dari pemanggilan fungsi menu_utama() yang menampilkan pilihan Login, Register, dan Keluar. Fungsi ini mengembalikan angka pilihan, atau None jika input bukan angka. Pilihan 1 memanggil proses_login() dan masuk ke dashboard() bila login berhasil. Pilihan 2 memanggil proses_register(), sedangkan pilihan 3 mengakhiri program. Input selain itu menampilkan pesan kesalahan, lalu menu ditampilkan kembali. Setiap pemanggilan fungsi digambar dengan simbol Predefined Process.



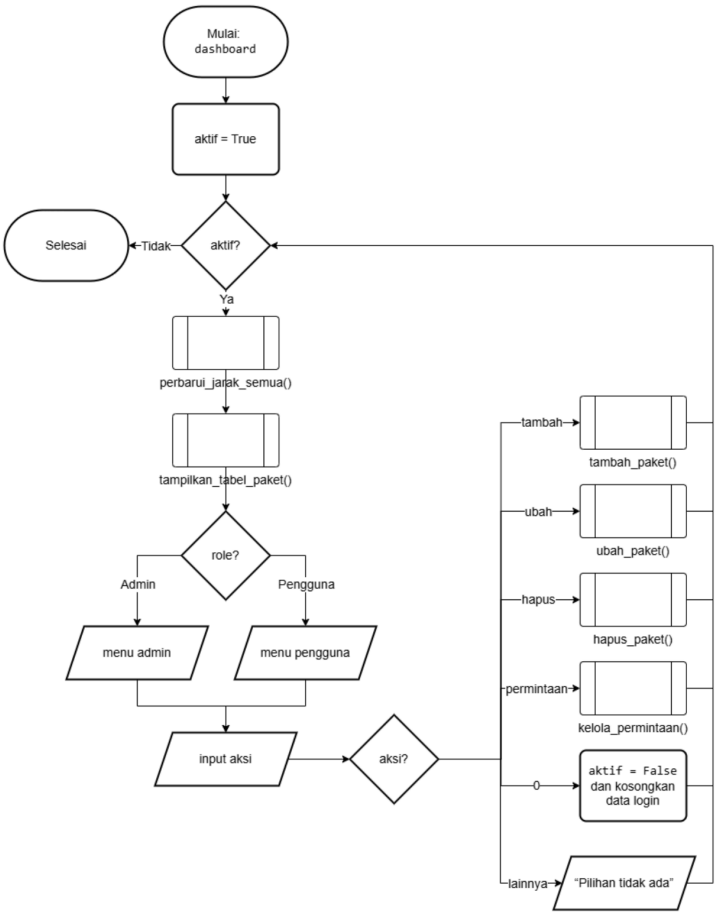
Gambar 1.2 Flowchart proses_login()

Fungsi proses_login() meminta username dan password, lalu memeriksa tiga kondisi: input kosong, username tidak terdaftar, dan password tidak cocok. Jika salah satunya terjadi, ditampilkan pesan kesalahan dan pengguna ditawari membuat akun dengan memanggil proses_register(), kemudian fungsi mengembalikan False. Jika semua benar, username dan role disimpan ke variabel global dan fungsi mengembalikan True.



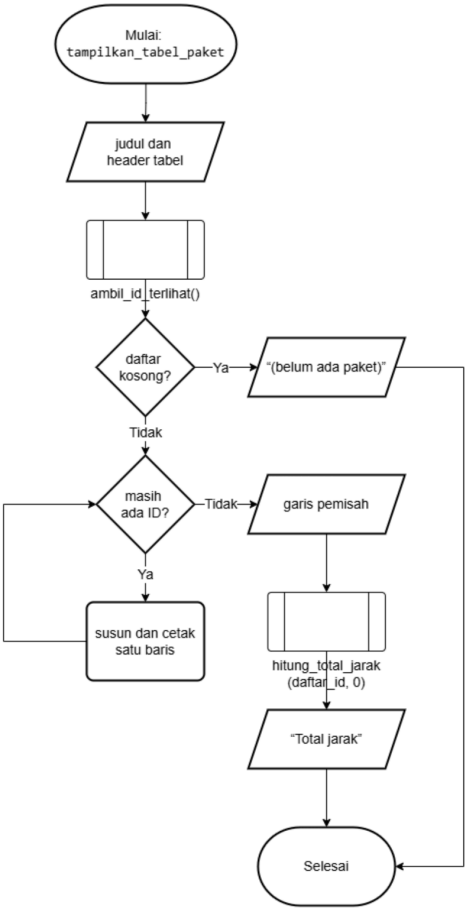
Gambar 1.3 Flowchart proses_register()

Prosedur proses_register() meminta username dan password. Jika salah satunya kosong atau username sudah dipakai, pendaftaran dibatalkan dengan pesan kesalahan. Jika valid, akun baru dengan role pengguna disimpan ke user_data dan ditampilkan pesan registrasi berhasil.



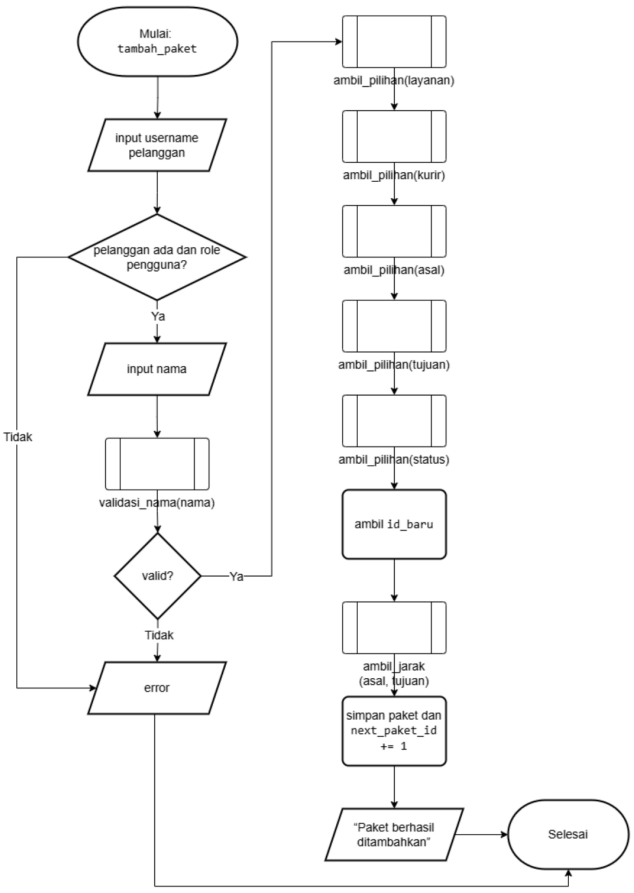
Gambar 1.4 Flowchart dashboard()

Prosedur dashboard() berisi perulangan selama pengguna belum logout. Setiap putaran memanggil perbarui_jarak_semua() dan tampilkan_tabel_paket(), menampilkan menu sesuai role (admin atau pengguna), lalu membaca pilihan. Pilihan akan memanggil tambah_paket(), ubah_paket(), hapus_paket(), atau kelola_permintaan(). Pilihan 0 menghentikan perulangan dan mengosongkan data login.



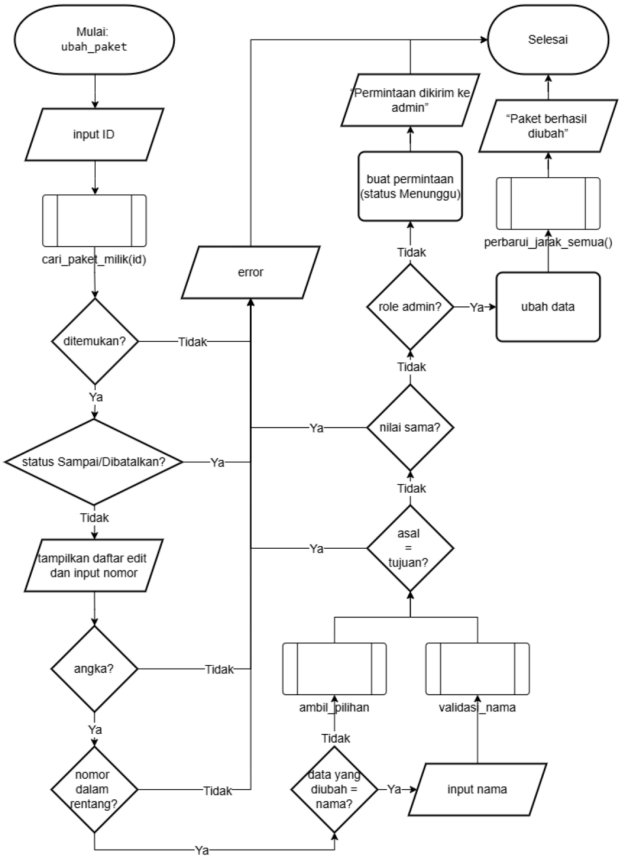
Gambar 1.5 Flowchart tampilkan_tabel_paket()

Prosedur ini mencetak judul dan header tabel, mengambil daftar ID paket yang boleh dilihat lewat ambil_id_terlihat(), lalu mencetak tiap paket dalam perulangan. Jika daftar kosong, ditampilkan keterangan belum ada paket. Jika ada paket, ditampilkan total jarak seluruh paket yang dihitung oleh fungsi rekursif hitung_total_jarak().



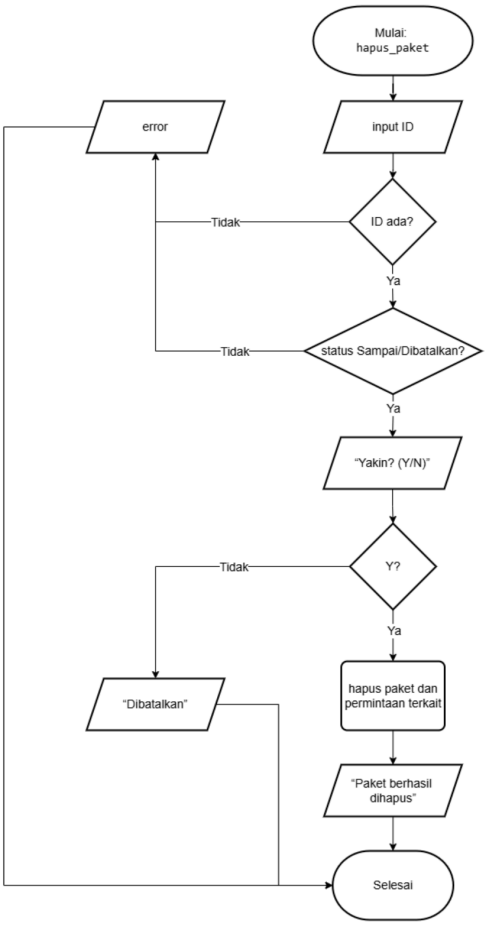
Gambar 1.6 Flowchart tambah_paket()

Pada prosedur ini admin memasukkan username pelanggan dan nama paket. Pelanggan harus terdaftar dengan role pengguna, dan nama divalidasi oleh validasi_nama(). Setelah itu layanan, kurir, asal, tujuan, dan status dipilih menggunakan ambil_pilihan(). ID baru dibuat, jarak dihitung dengan ambil_jarak(), dan paket disimpan. Setiap kesalahan menampilkan pesan lalu prosedur selesai.



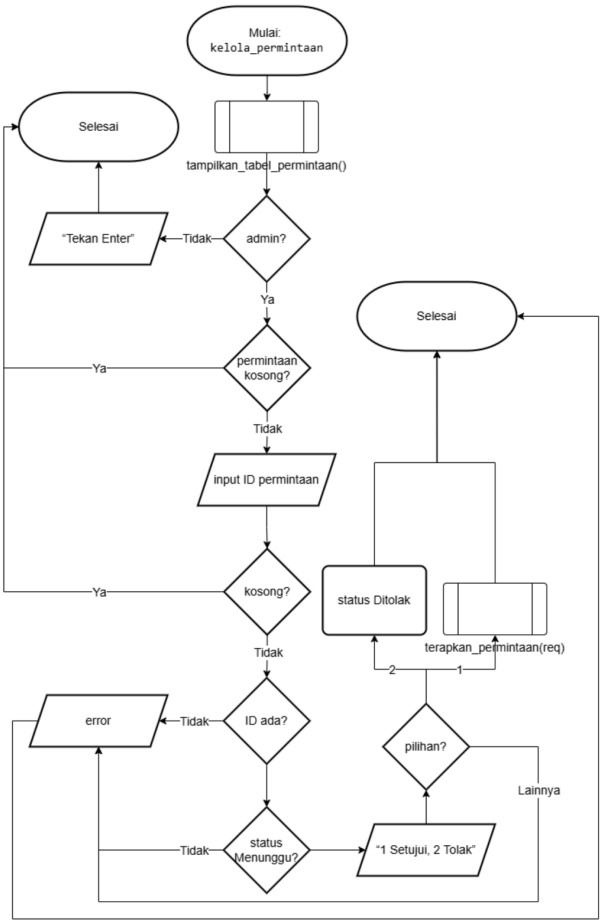
Gambar 1.7 Flowchart ubah_paket()

Prosedur ini dipakai admin untuk mengubah paket dan pengguna untuk mengajukan perubahan. Paket dicari dengan cari_paket_milik(), dan paket berstatus Sampai atau Dibatalkan tidak dapat diubah. Pengguna memilih data yang diubah, memasukkan nilai baru yang divalidasi, lalu program memeriksa lokasi asal dan tujuan tidak sama serta nilai baru berbeda dari nilai sekarang. Admin langsung mengubah data dan memperbarui jarak, sedangkan pengguna membuat permintaan berstatus Menunggu.



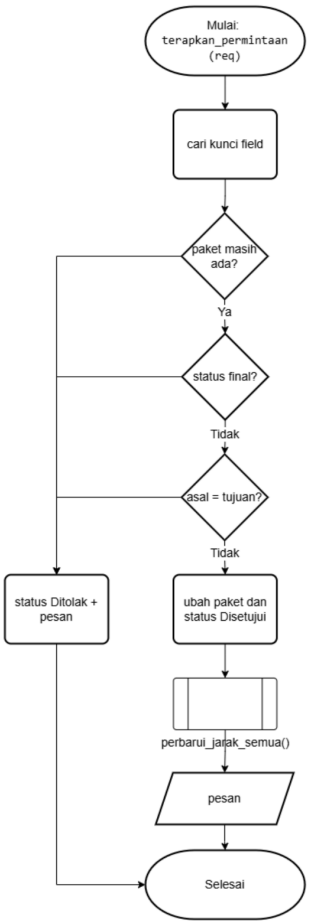
Gambar 1.8 Flowchart hapus_paket()

Prosedur ini memeriksa apakah ID paket ada dan statusnya Sampai atau Dibatalkan. Jika memenuhi, program meminta konfirmasi. Jika dijawab Y, paket beserta permintaan terkait dihapus. Jika dijawab selain Y, penghapusan dibatalkan.



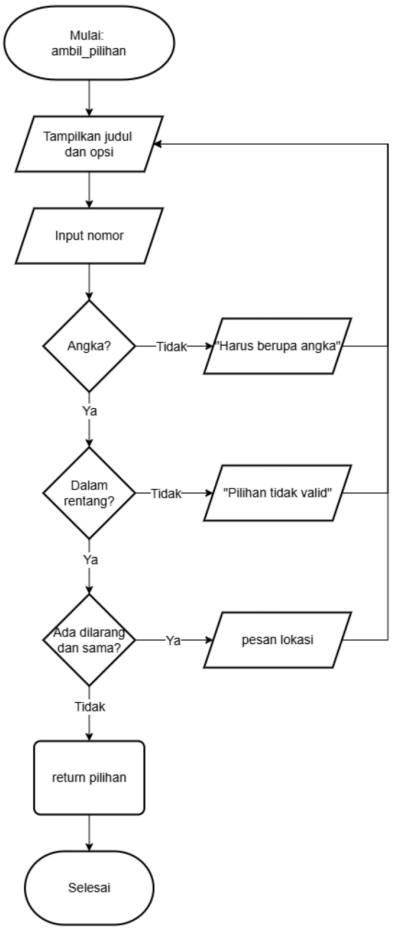
Gambar 1.9 Flowchart kelola_permintaan()

Prosedur ini menampilkan tabel permintaan lewat tampilkan_tabel_permintaan(). Pengguna hanya melihat riwayat permintaannya sendiri. Admin dapat memilih ID permintaan yang berstatus Menunggu, lalu menyetujui (memanggil terapkan_permintaan()) atau menolaknya.



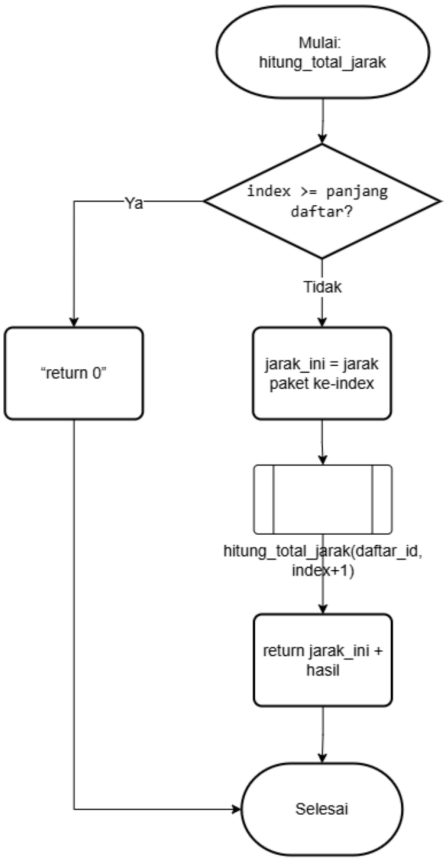
Gambar 1.10 Flowchart terapkan_permintaan(req)

Prosedur dengan parameter ini menerapkan permintaan yang disetujui admin. Permintaan otomatis ditolak jika paket sudah tidak ada, statusnya sudah final, atau lokasi asal sama dengan tujuan. Jika lolos, data paket diubah, status permintaan menjadi Disetujui, dan jarak diperbarui.



Gambar 1.11 Flowchart ambil_pilihan()

Fungsi ini menampilkan daftar opsi dan meminta nomor dalam perulangan. Input bukan angka, nomor di luar rentang, dan lokasi tujuan yang sama dengan asal masing-masing menampilkan pesan kesalahan, lalu perulangan diulang. Jika valid, fungsi mengembalikan opsi yang dipilih.



Gambar 1.12 Flowchart hitung_total_jarak() (rekursif)

Fungsi rekursif ini menjumlahkan jarak paket dari posisi index sampai akhir daftar. Kasus dasarnya adalah ketika index sudah mencapai panjang daftar, fungsi mengembalikan 0. Jika belum, fungsi mengambil jarak paket pada index lalu menjumlahkannya dengan hasil pemanggilan dirinya sendiri untuk index berikutnya.

2. Deskripsi Singkat Program

Program ini adalah Sistem Manajemen Pengiriman Paket. Program menyediakan fitur autentikasi melalui Login dan Register, serta pengelolaan paket yang mencakup penambahan, penampilan, perubahan, dan penghapusan data. Program membedakan hak akses admin dan pengguna biasa (multiuser), sehingga data yang dapat dilihat atau diubah mengikuti peran masing-masing. Pengguna biasa dapat mengajukan permintaan perubahan paket yang kemudian disetujui atau ditolak oleh admin.

3. Source Code

A. Variabel Global dan Struktur Data

Bagian ini berisi variabel global yang dipakai seluruh fungsi, yaitu data pengguna, data paket, data permintaan, penghitung ID, serta data login yang sedang aktif.

```
# ==================== VARIABEL GLOBAL ====================

user_data = {
    "admin":    {"password": "admin1",    "role": "admin"},
    "pengguna": {"password": "pengguna1", "role": "pengguna"}
}

layanan_data = ["Reguler", "Express", "Same Day"]
kurir_data = ["Budi", "Sinta", "Andi"]
lokasi_data = ["Balikpapan", "Samarinda", "Bontang", "Tenggarong", "Penajam"]
status_data = ["Diproses", "Dikirim", "Sampai", "Dibatalkan"]

jarak_data = {
    "Balikpapan": {"Samarinda": 115, "Bontang": 235, "Tenggarong": 130, "Penajam": 40},
    "Samarinda":  {"Balikpapan": 115, "Bontang": 120, "Tenggarong": 35, "Penajam": 150},
    "Bontang":    {"Balikpapan": 235, "Samarinda": 120, "Tenggarong": 155, "Penajam": 275},
    "Tenggarong": {"Balikpapan": 130, "Samarinda": 35, "Bontang": 155, "Penajam": 165},
    "Penajam":    {"Balikpapan": 40, "Samarinda": 150, "Bontang": 275, "Tenggarong": 165}
}

paket_data = {
    "1": {"pemilik": "pengguna", "nama": "Laptop", "layanan": "Express",
          "kurir": "Budi", "asal": "Balikpapan", "tujuan": "Samarinda",
          "jarak": 0, "status": "Dikirim"}
}

permintaan_data = {}

edit_fields = [
    ["Nama paket", "nama", None],
    ["Jenis layanan", "layanan", layanan_data],
    ["Kurir", "kurir", kurir_data],
```

```
    ["Lokasi asal", "asal", lokasi_data],
    ["Lokasi tujuan", "tujuan", lokasi_data],
    ["Status", "status", status_data]
]

next_paket_id = 2
next_req_id = 1
username_aktif = ""
role_aktif = ""
clear = "cls" if os.name == "nt" else "clear"
```

B. Exception Buatan Sendiri

Tiga class exception dibuat untuk membedakan jenis kesalahan input pada error handling.

```
# ==================== ERROR (EXCEPTION) KUSTOM ====================

class NamaPaketError(Exception):
    """Dilempar jika nama paket kosong atau lebih dari 15 karakter."""
    pass


class LokasiSamaError(Exception):
    """Dilempar jika lokasi asal sama dengan lokasi tujuan."""
    pass


class AkunError(Exception):
    """Dilempar jika data login / register tidak valid."""
    pass
```

C. Prosedur Tampilan

Prosedur untuk membersihkan layar, menampilkan pesan, menampilkan tabel paket dan permintaan, serta memperbarui jarak.

```
def bersihkan_layar():
    os.system(clear)


def tampilkan_pesan(pesan, detik):
    print(pesan)
    time.sleep(detik)


def tampilkan_tabel_paket():
    header = ["ID", "Pemilik", "Paket", "Layanan", "Kurir", "Asal", "Tujuan", "Jarak", "Status"]
    kolom = ["pemilik", "nama", "layanan", "kurir", "asal", "tujuan", "jarak", "status"]
    widths = [8, 10, 16, 10, 8, 12, 12, 9, 11]

    print(f"Login sebagai: {username_aktif} ({role_aktif})")
    print()
    print("DAFTAR SEMUA PAKET" if role_aktif == "admin" else "PAKET SAYA")

    baris = ""
    for i in range(len(header)):
        baris += header[i].ljust(widths[i])
    print(baris)
    print("-" * sum(widths))
```

```
    daftar_id = ambil_id_terlihat()
    for id_p in daftar_id:
        paket = paket_data[id_p]
        baris = id_p.ljust(widths[0])
        for i in range(len(kolom)):
            nilai = str(paket["jarak"]) + " km" if kolom[i] == "jarak" else str(paket[kolom[i]])
            baris += nilai.ljust(widths[i + 1])
        print(baris)

    if len(daftar_id) == 0:
        print("(belum ada paket)")
    else:
        print("-" * sum(widths))
        print(f"Total jarak seluruh paket: {hitung_total_jarak(daftar_id, 0)} km")


def tampilkan_tabel_permintaan():
    header = ["ID", "Pelanggan", "Paket", "Perubahan", "Nilai baru", "Status"]
    kolom = ["pelanggan", "paket", "perubahan", "nilai_baru", "status"]
    widths = [8, 12, 9, 16, 14, 10]

    baris = ""
    for i in range(len(header)):
        baris += header[i].ljust(widths[i])
    print(baris)
    print("-" * sum(widths))

    jumlah = 0
    for id_r in permintaan_data:
        r = permintaan_data[id_r]
        if role_aktif == "admin" or r["pelanggan"] == username_aktif:
            baris = id_r.ljust(widths[0])
            for i in range(len(kolom)):
                baris += str(r[kolom[i]]).ljust(widths[i + 1])
            print(baris)
            jumlah += 1

    if jumlah == 0:
        print("(belum ada permintaan)")
    print()


def perbarui_jarak_semua():
    for id_p in paket_data:
        try:
            paket_data[id_p]["jarak"] = ambil_jarak(paket_data[id_p]["asal"],
                                                    paket_data[id_p]["tujuan"])
        except KeyError:
            paket_data[id_p]["jarak"] = 0
            print(f"Peringatan: data jarak paket {id_p} tidak ditemukan.")
```

D. Fitur Register dan Login

proses_register() adalah prosedur, sedangkan proses_login() adalah fungsi tanpa parameter yang mengembalikan True atau False.

```
def proses_register():
    print("Register")
    username = input("Masukkan username: ").strip()
    password = input("Masukkan password: ").strip()
    try:
```

```
        if username == "" or password == "":
            raise AkunError("Username dan password tidak boleh kosong.")
        if username in user_data:
            raise AkunError("Username sudah dipakai.")
        user_data[username] = {"password": password, "role": "pengguna"}
        print("Registrasi berhasil!")
    except AkunError as e:
        print(e)
    finally:
        time.sleep(1)
        bersihkan_layar()

def proses_login():
    global username_aktif, role_aktif
    print("Login")
    username = input("Masukkan username: ").strip()
    password = input("Masukkan password: ").strip()
    try:
        if username == "" or password == "":
            raise AkunError("Username dan password tidak boleh kosong.")
        if username not in user_data:
            raise AkunError("Username salah.")
        if user_data[username]["password"] != password:
            raise AkunError("Password salah.")
    except AkunError as e:
        tampilkan_pesan(e, 1)
        bersihkan_layar()
        if input("Tidak punya akun, apakah ingin membuat akun? (Y/N): ").upper() == "Y":
            bersihkan_layar()
            proses_register()
        return False
```

E. Fitur Tambah Data (Create)

Prosedur untuk menambahkan paket baru oleh admin.

```
def tambah_paket():
    global next_paket_id
    print("TAMBAH PAKET")
    pemilik = input("Username pelanggan: ").strip()
    try:
        if pemilik not in user_data or user_data[pemilik]["role"] != "pengguna":
            raise KeyError(pemilik)
        nama = validasi_nama(input("Nama paket (maks 15 karakter): ").strip())
        layanan = ambil_pilihan("Jenis layanan", layanan_data)
        kurir = ambil_pilihan("Kurir", kurir_data)
        asal = ambil_pilihan("Lokasi asal", lokasi_data)
        tujuan = ambil_pilihan("Lokasi tujuan", lokasi_data, asal)
        status = ambil_pilihan("Status", status_data)
    except KeyError:
        print("Pelanggan tidak ditemukan.")
    except NamaPaketError as e:
        print(e)
    else:
        id_baru = str(next_paket_id)
        paket_data[id_baru] = {"pemilik": pemilik, "nama": nama, "layanan": layanan,
                               "kurir": kurir, "asal": asal, "tujuan": tujuan,
                               "jarak": ambil_jarak(asal, tujuan), "status": status}
        next_paket_id += 1
        print(f"Paket {id_baru} berhasil ditambahkan!")
    finally:
        time.sleep(1.5)
```

F. Fitur Ubah Data (Update)

Prosedur untuk mengubah paket (admin) atau mengajukan perubahan (pengguna).

```
def ubah_paket():
    global next_req_id
    print("UBAH PAKET" if role_aktif == "admin" else "AJUKAN PERUBAHAN PAKET")
    id_paket = input("Masukkan ID paket: ").strip().upper()

    try:
        target = cari_paket_milik(id_paket)
        if target["status"] in ("Sampai", "Dibatalkan"):
            raise ValueError(f"Paket sudah {target['status']}, tidak bisa diubah.")

        print("Data yang ingin diubah:")
        for n, editf in enumerate(edit_fields, 1):
            print(f"  {n}. {editf[0]}")
        try:
            nomor = int(input("Pilih nomor: "))
        except ValueError:
            print("Input harus berupa angka.")
            return
        if nomor < 1:
            raise IndexError
        editf = edit_fields[nomor - 1]
        if editf[2] is None:
            baru = validasi_nama(input("Nama paket baru (maks 15 karakter): ").strip())
        else:
            baru = ambil_pilihan(editf[0] + " baru", editf[2])

        asal_baru = baru if editf[1] == "asal" else target["asal"]
        tujuan_baru = baru if editf[1] == "tujuan" else target["tujuan"]
        if asal_baru == tujuan_baru:
            raise LokasiSamaError("Lokasi asal dan tujuan tidak boleh sama.")
        if baru == target[editf[1]]:
            raise ValueError("Nilai baru sama dengan nilai sekarang.")

        if role_aktif == "admin":
            target[editf[1]] = baru
            perbarui_jarak_semua()
            print("Paket berhasil diubah! Jarak diperbarui otomatis.")
        else:
            id_req = f"REQ{next_req_id:03d}"
            permintaan_data[id_req] = {"pelanggan": username_aktif, "paket": id_paket,
                                       "perubahan": editf[0], "nilai_baru": baru,
                                       "status": "Menunggu"}
            next_req_id += 1
            print("Permintaan perubahan dikirim ke admin.")
    except KeyError:
        print("Paket tidak ditemukan.")
    except (NamaPaketError, LokasiSamaError) as e:
        print(e)
    except IndexError:
        print("Pilihan tidak valid.")
    except ValueError as e:
        print(e)
    finally:
        time.sleep(1.5)
```

G. Fitur Hapus Data (Delete)

Prosedur untuk menghapus paket berdasarkan ID.

```
def hapus_paket():
    print("HAPUS PAKET")
    id_paket = input("Masukkan ID paket: ").strip().upper()
    try:
        if id_paket not in paket_data:
            raise KeyError(id_paket)
        if paket_data[id_paket]["status"] not in ("Sampai", "Dibatalkan"):
            raise ValueError("Paket hanya bisa dihapus jika statusnya Sampai atau Dibatalkan.")

        yakin = input("Yakin ingin menghapus? (Y/N): ")
        if yakin.upper() == "Y":
            del paket_data[id_paket]
            id_hapus = [i for i in permintaan_data if permintaan_data[i]["paket"] == id_paket]
            for id_r in id_hapus:
                del permintaan_data[id_r]
            print("Paket berhasil dihapus!")
        else:
            print("Dibatalkan.")
    except KeyError:
        print("Paket tidak ditemukan.")
    except ValueError as e:
        print(e)
    finally:
        time.sleep(1.5)
```

H. Fitur Permintaan Perubahan

Prosedur untuk menampilkan dan memproses permintaan perubahan paket.

```
def kelola_permintaan():
    print("PERMINTAAN PERUBAHAN")
    tampilkan_tabel_permintaan()

    if role_aktif != "admin":
        input("Tekan Enter untuk kembali...")
        return
    if len(permintaan_data) == 0:
        time.sleep(1.5)
        return

    id_req = input("ID permintaan yang diproses (kosongkan untuk kembali): ").strip().upper()
    if id_req == "":
        return

    try:
        req = permintaan_data[id_req]
        if req["status"] != "Menunggu":
            raise ValueError("Permintaan ini sudah diproses.")

        proses = input("1. Setujui  2. Tolak : ")
        if proses == "2":
            req["status"] = "Ditolak"
            print("Permintaan ditolak.")
        elif proses == "1":
            terapkan_permintaan(req)
```

```
        else:
            print("Pilihan tidak valid.")
    except KeyError:
        print("Permintaan tidak ditemukan.")
    except ValueError as e:
        print(e)
    finally:
        time.sleep(1.5)


def terapkan_permintaan(req):
    kunci = ""
    for editf in edit_fields:
        if editf[0] == req["perubahan"]:
            kunci = editf[1]

    try:
        target = paket_data[req["paket"]]
        if target["status"] in ("Sampai", "Dibatalkan"):
            raise ValueError("Paket sudah tidak bisa diubah. Permintaan ditolak.")

        asal_baru = req["nilai_baru"] if kunci == "asal" else target["asal"]
        tujuan_baru = req["nilai_baru"] if kunci == "tujuan" else target["tujuan"]
        if asal_baru == tujuan_baru:
            raise LokasiSamaError("Lokasi asal dan tujuan sama. Permintaan ditolak.")

        target[kunci] = req["nilai_baru"]
        req["status"] = "Disetujui"
        perbarui_jarak_semua()
        print("Permintaan disetujui! Jarak diperbarui otomatis.")
    except KeyError:
        req["status"] = "Ditolak"
        print("Paket sudah tidak ada. Permintaan ditolak.")
    except (ValueError, LokasiSamaError) as e:
        req["status"] = "Ditolak"
        print(e)
```

I. Fungsi Tanpa Parameter

Fungsi yang mengembalikan nilai tanpa menerima parameter.

```
def menu_utama():
    print("1. Login")
    print("2. Register")
    print("3. Keluar")
    try:
        return int(input("Pilih menu: "))
    except ValueError:
        return None


def ambil_id_terlihat():
    hasil = []
    for id_p in paket_data:
        if role_aktif == "admin" or paket_data[id_p]["pemilik"] == username_aktif:
            hasil.append(id_p)
    return hasil
```

J. Fungsi Dengan Parameter

Fungsi yang menerima parameter dan mengembalikan nilai.

```
def ambil_jarak(asal, tujuan):
    return jarak_data[asal][tujuan]


def validasi_nama(nama):
    if nama == "":
        raise NamaPaketError("Nama paket tidak boleh kosong.")
    if len(nama) > 15:
        raise NamaPaketError("Nama paket tidak lebih dari 15 karakter.")
    return nama


def cari_paket_milik(id_paket):
    paket = paket_data[id_paket]
    if role_aktif != "admin" and paket["pemilik"] != username_aktif:
        raise KeyError(id_paket)
    return paket


def ambil_pilihan(judul, opsi, dilarang=None):
    while True:
        print(judul + ":")
        for n, opt in enumerate(opsi, 1):
            print(f"  {n}. {opt}")
        try:
            nomor = int(input("Pilih nomor: "))
            if nomor < 1 or nomor > len(opsi):
                raise IndexError
            pilihan = opsi[nomor - 1]
            if dilarang is not None and pilihan == dilarang:
                raise LokasiSamaError("Lokasi tujuan tidak boleh sama dengan lokasi asal.")
            return pilihan
        except ValueError:
            print("Input harus berupa angka.")
        except IndexError:
            print("Pilihan tidak valid.")
        except LokasiSamaError as e:
            print(e)
```

K. Fungsi Rekursif

Fungsi yang memanggil dirinya sendiri untuk menjumlahkan jarak paket. Kasus dasarnya adalah index mencapai panjang daftar.

```
def hitung_total_jarak(daftar_id, index):
    if index >= len(daftar_id):
        return 0
    jarak_ini = paket_data[daftar_id[index]]["jarak"]
    return jarak_ini + hitung_total_jarak(daftar_id, index + 1)
```

L. Dashboard dan Program Utama

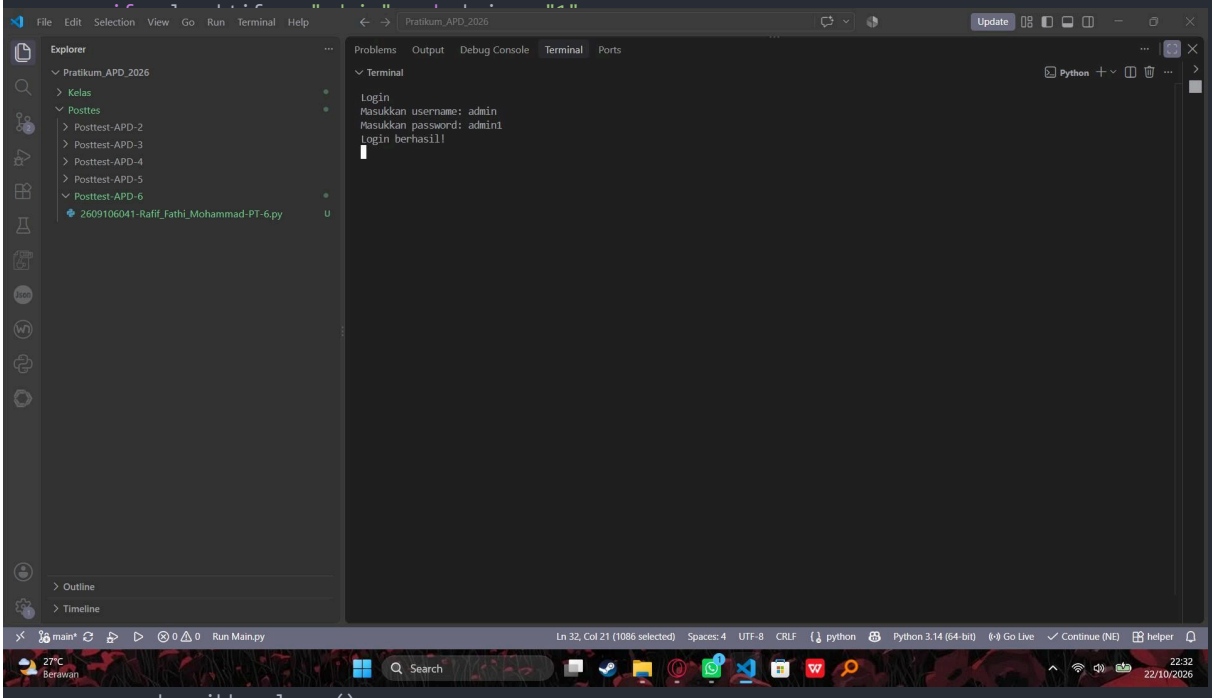
dashboard() mengatur menu setelah login, sedangkan main() mengatur menu awal. Seluruh program dibungkus try dan except untuk menangani KeyboardInterrupt dan EOFError.

```
def dashboard():
```

```
    global username_aktif, role_aktif
    aktif = True
    while aktif:
        bersihkan_layar()
        perbarui_jarak_semua()
        tampilkan_tabel_paket()

        print()
        if role_aktif == "admin":
            print("1. Tambah paket")
            print("2. Ubah paket")
            print("3. Hapus paket")
            print("4. Permintaan perubahan dari pelanggan")
        else:
            print("1. Ajukan perubahan paket")
            print("2. Riwayat permintaan saya")
        print("0. Logout")

        aksi = input("Pilih: ").strip()
        print()
```

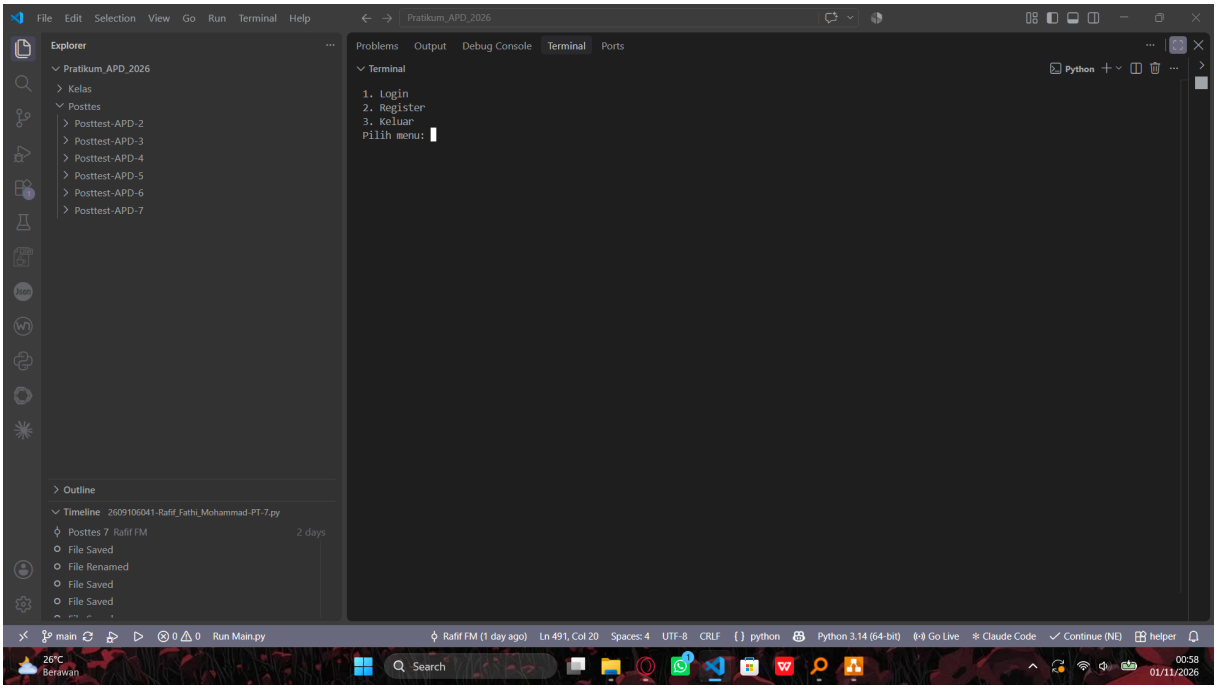



```
            bersihkan_layar()
            if proses_login():
                dashboard()
        elif pilih == 2:
            bersihkan_layar()
            proses_register()
        elif pilih == 3:
            program = False
            print("Terima kasih!")
        else:
            tampilkan_pesan("Invalid, menu tidak ada", 1)
        if program:
            bersihkan_layar()

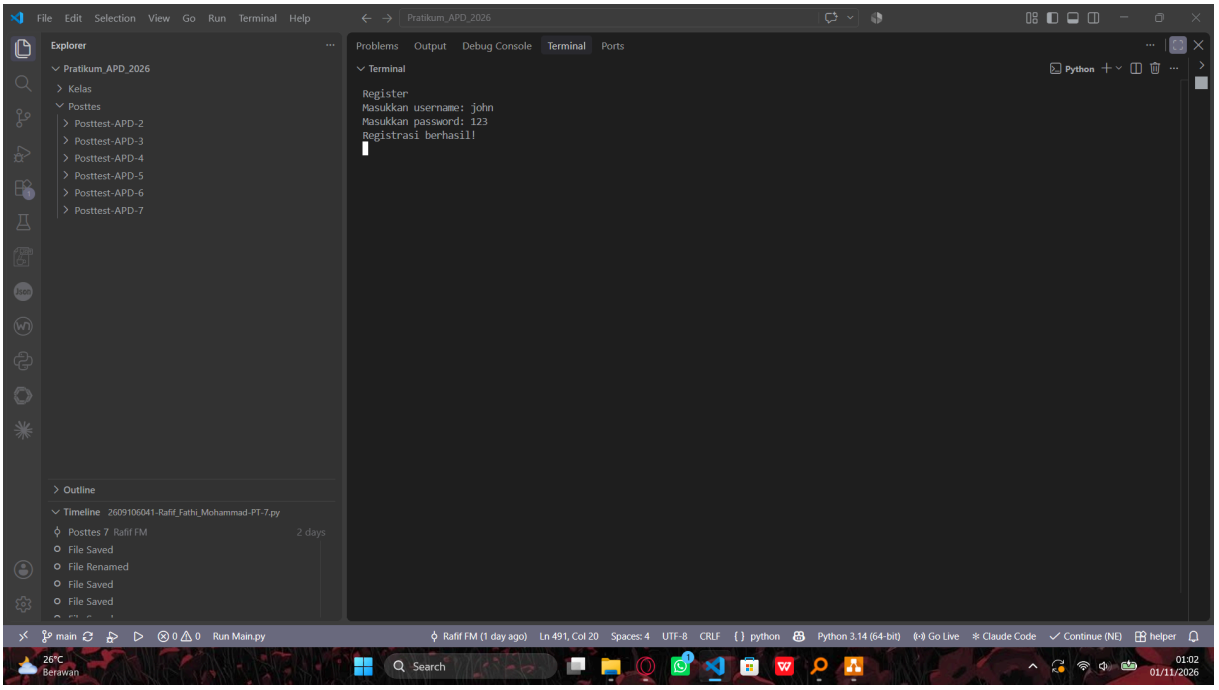

try:
    main()
except (KeyboardInterrupt, EOFError):
    print("\nProgram dihentikan oleh pengguna.")
```

```
finally:
    print("Sesi program selesai.")
```

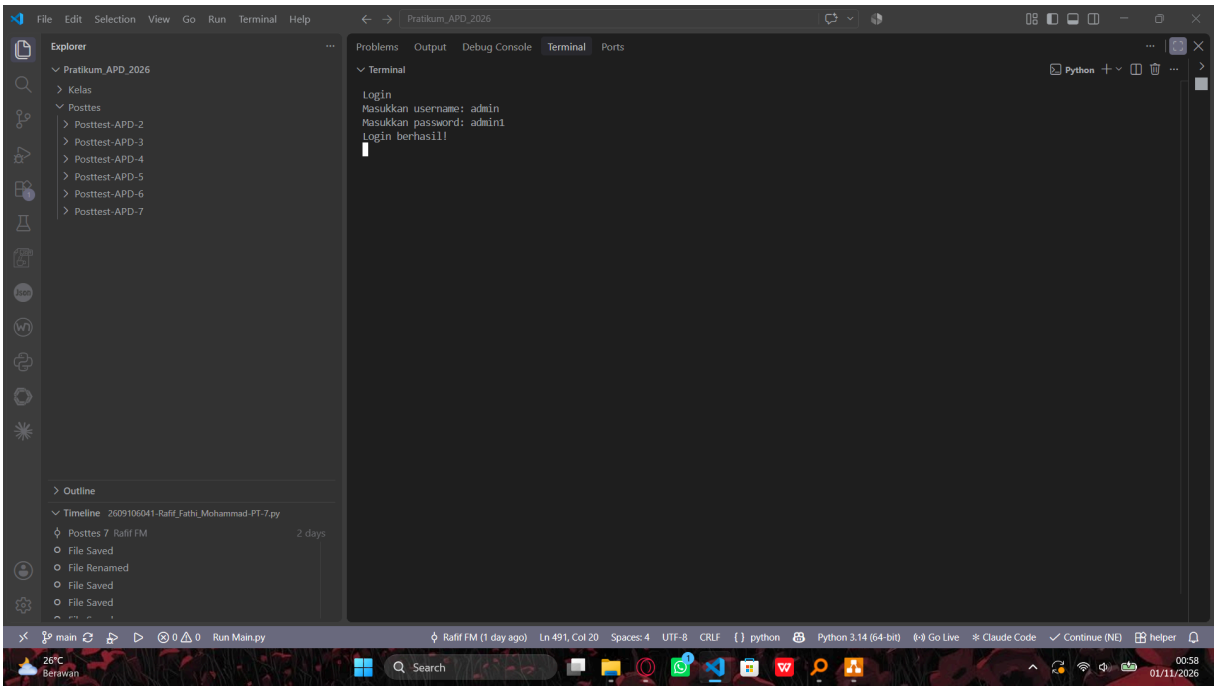
4. Hasil Output



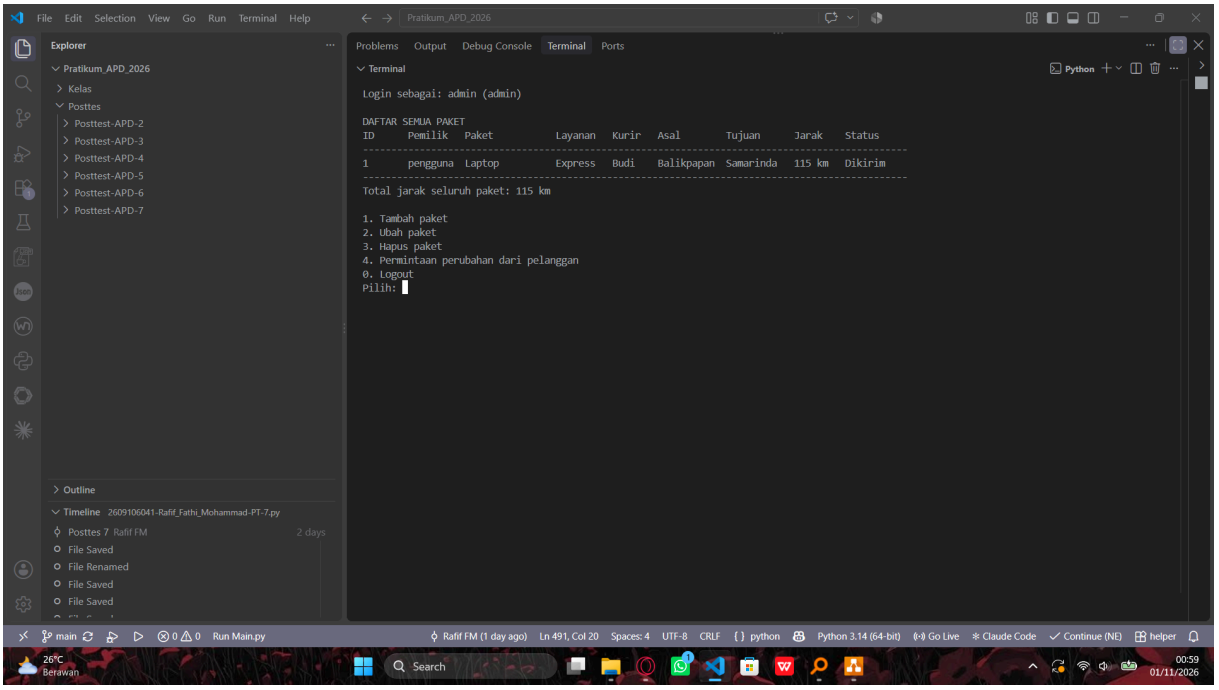
Gambar 4.1 Menu



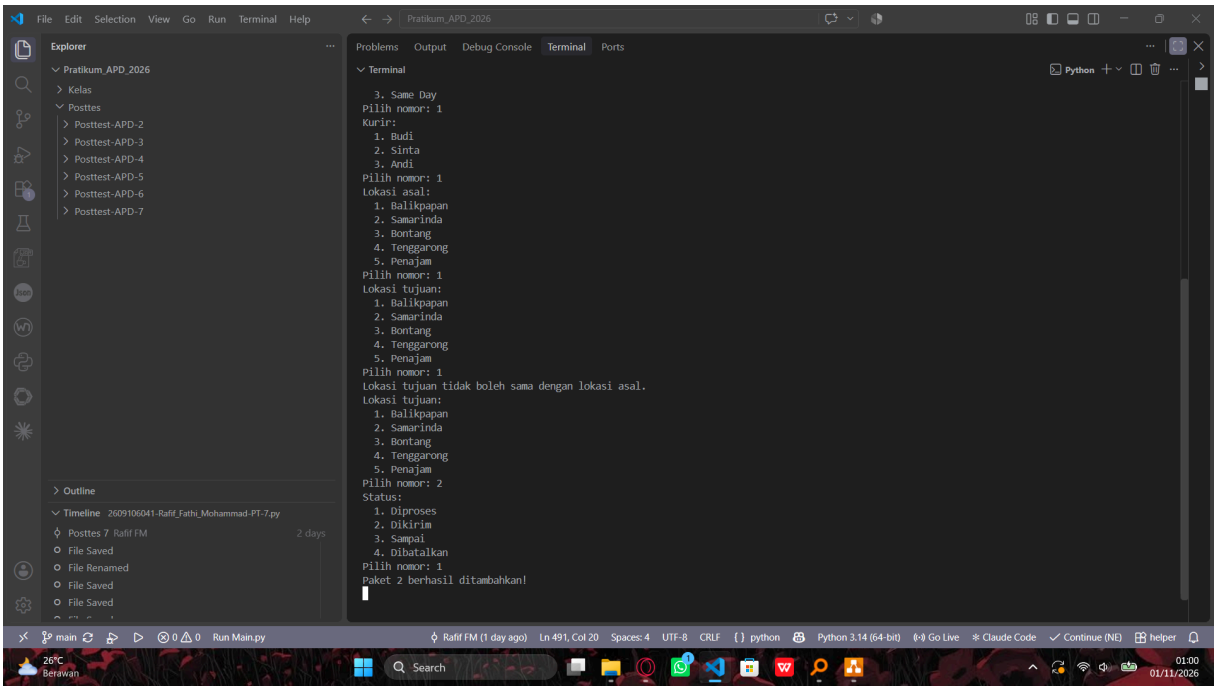
Gambar 4.2 Register



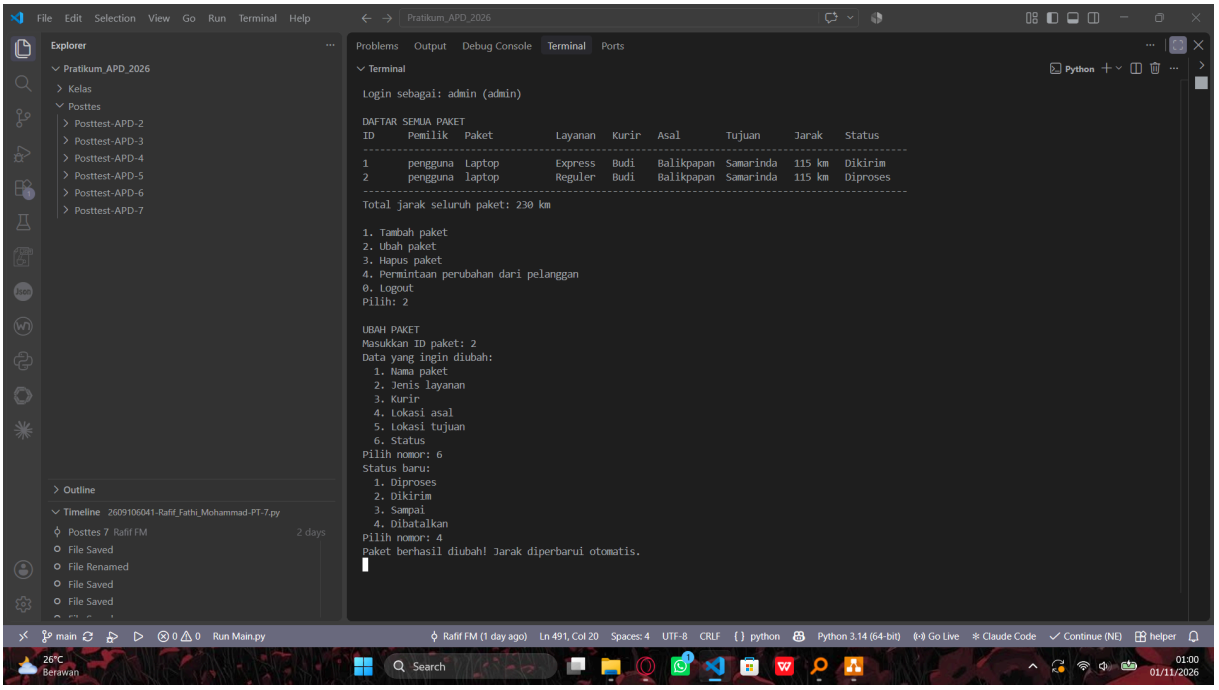
Gambar 4.3 Login



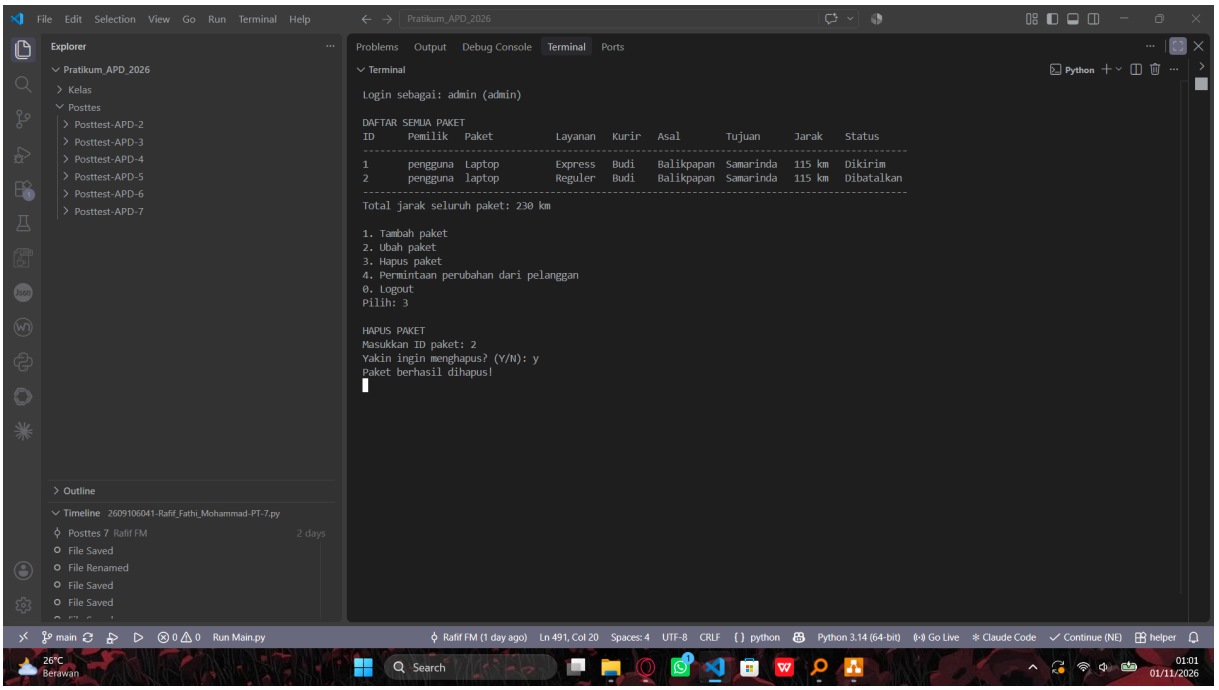
Gambar 4.4 Tabel Paket dan Total Jarak



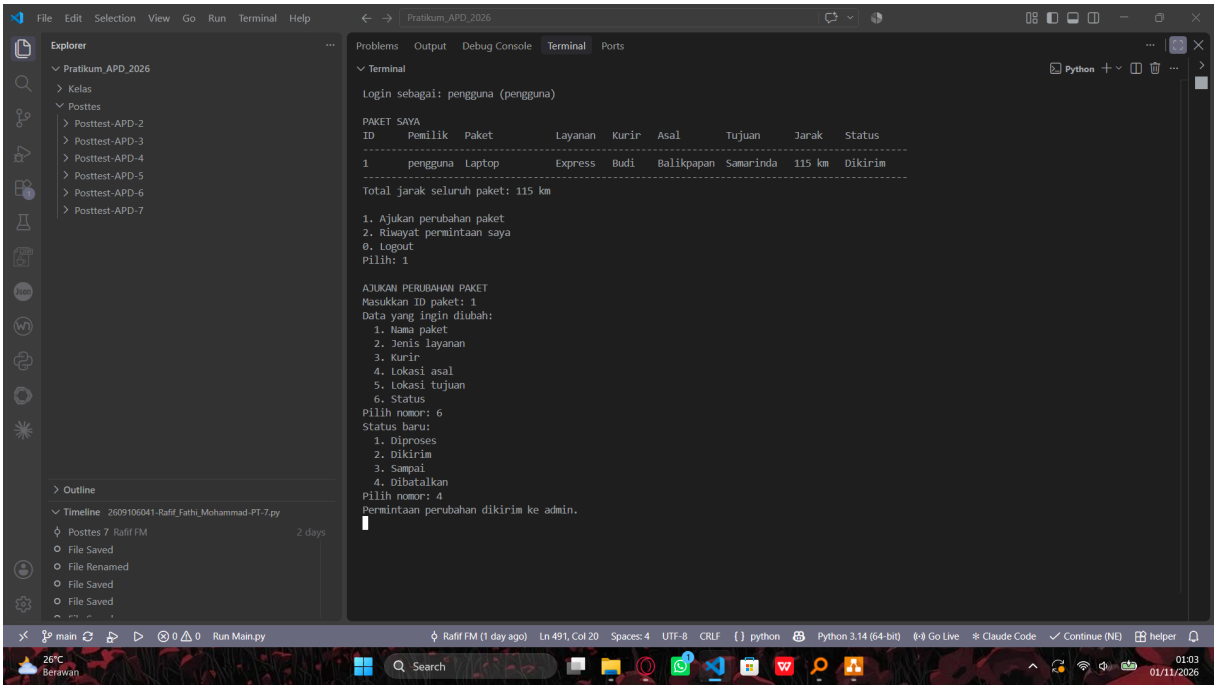
Gambar 4.5 Tambah Paket



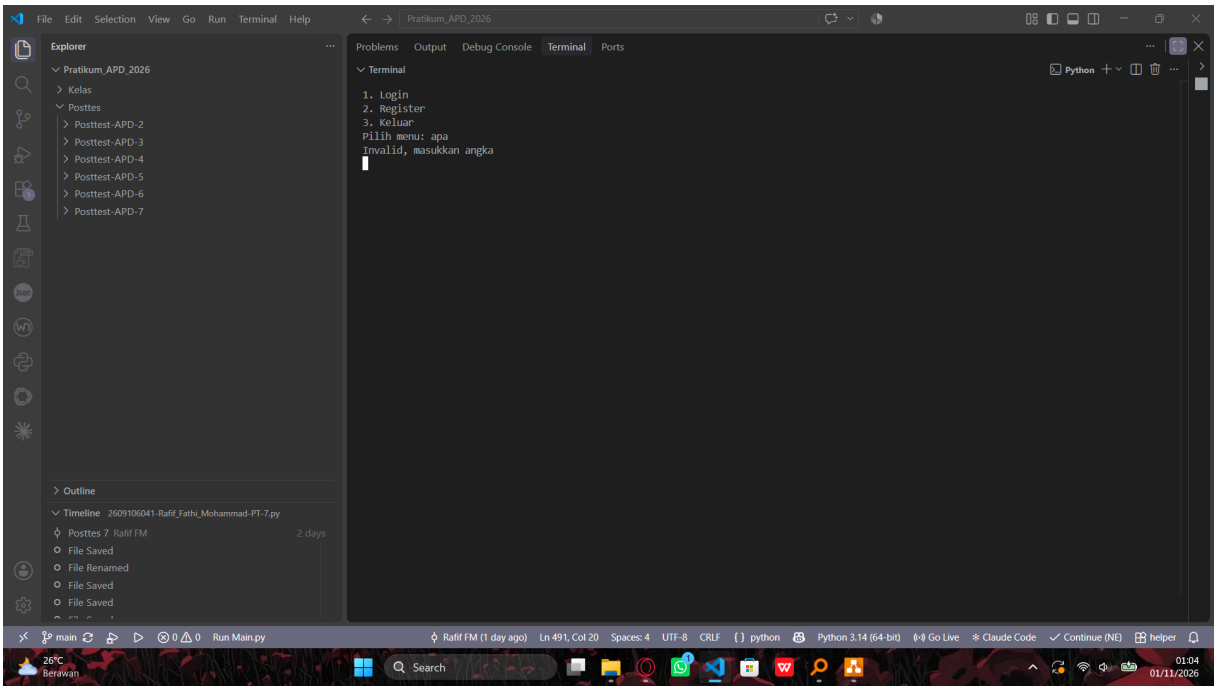
Gambar 4.6 Ubah Paket



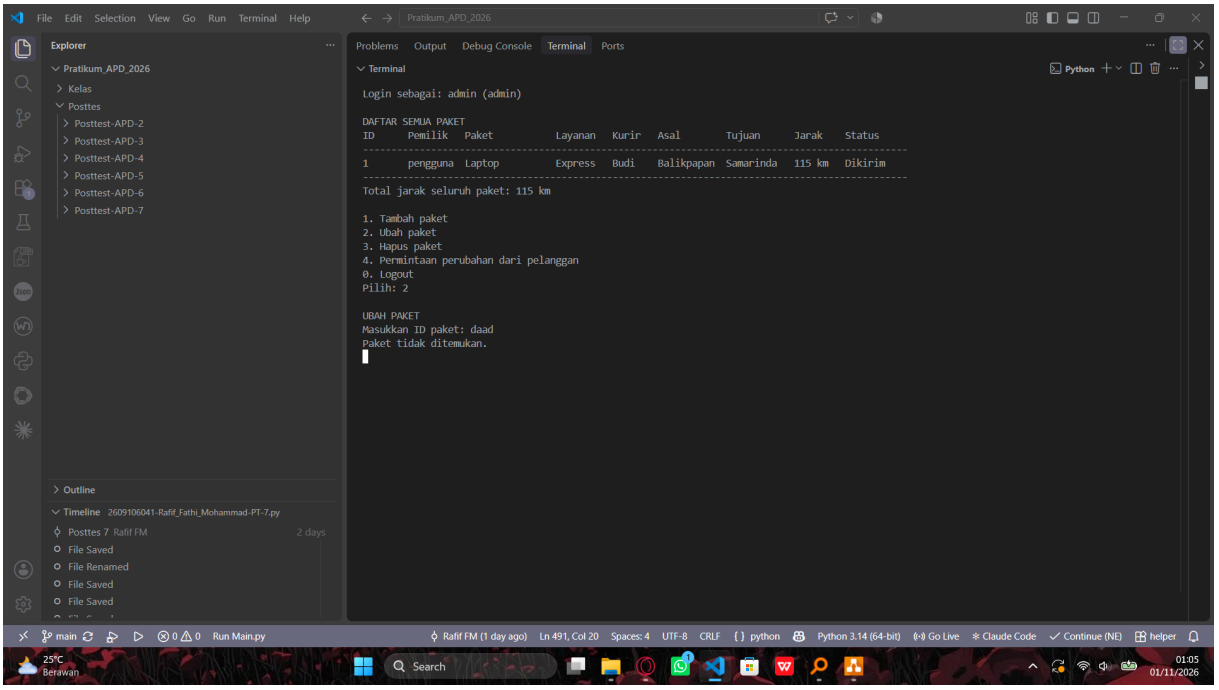
Gambar 4.7 Hapus Paket



Gambar 4.8 Ajukan Perubahan



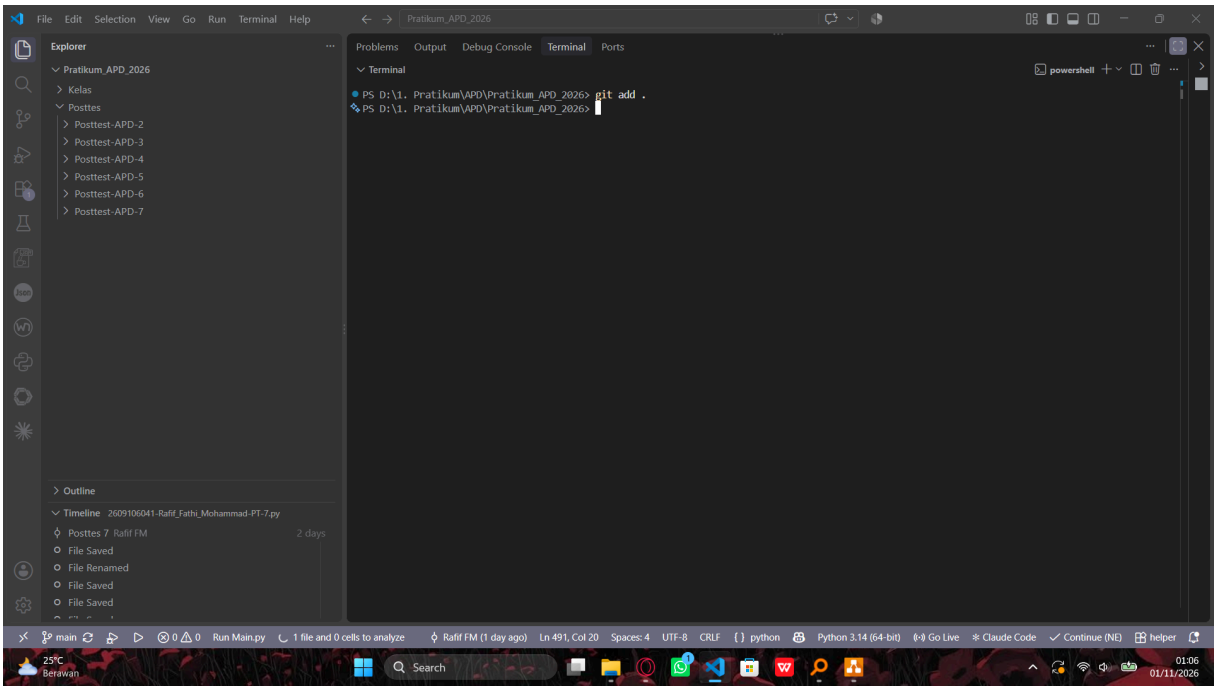
Gambar 4.9 Error Handling: Input Menu Bukan Angka



Gambar 4.10 Error Handling: Pilihan atau Nama Paket Tidak Valid

5. Langkah-langkah Git

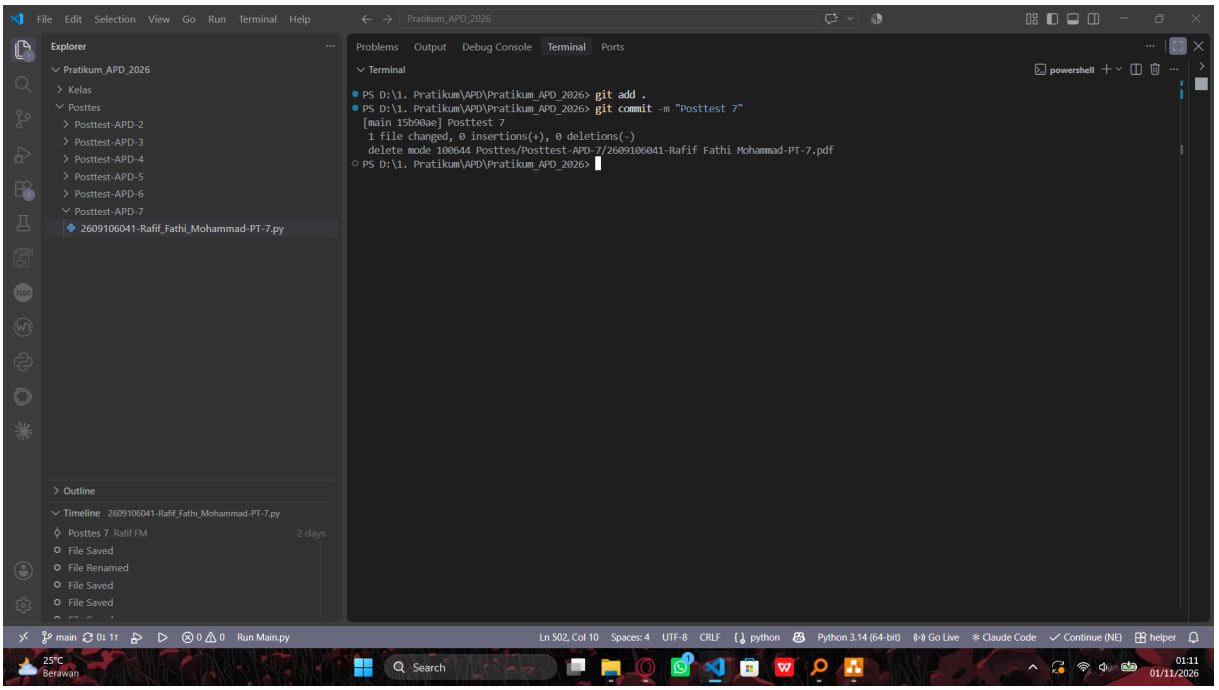
5.1 Git Add



Gambar 5.1 Git Add

Git Add digunakan untuk memasukkan file yang telah diubah ke dalam staging area. Tahap ini mempersiapkan file agar dapat disertakan dalam commit.

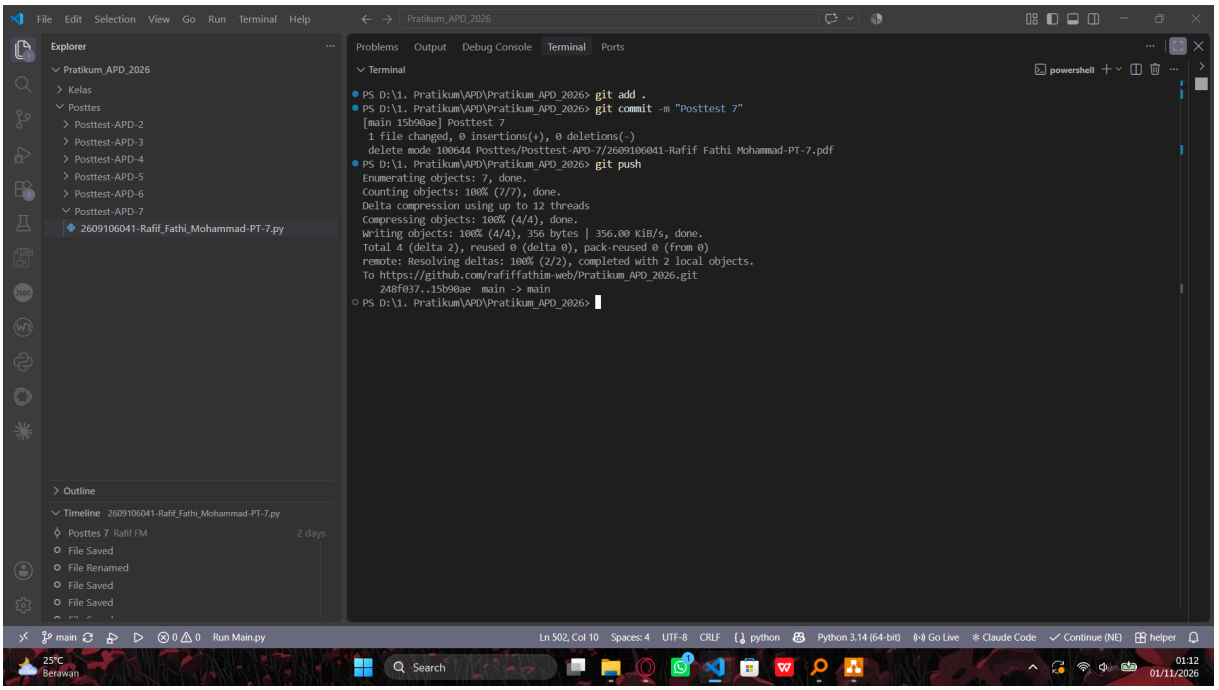
5.2 Git Commit



Gambar 5.2 Git Commit

Git Commit digunakan untuk menyimpan perubahan yang sudah dimasukkan ke staging area sebagai sebuah commit di repositori lokal. Pesan commit berfungsi menjelaskan perubahan yang dilakukan.

5.3 Git Push



Gambar 5.3 Git Push

Git Push digunakan untuk mengirimkan commit dari repositori lokal ke repositori jarak jauh, misalnya GitHub. Dengan demikian, perubahan yang telah disimpan dapat tersedia di repositori tersebut.